MOBILE DJ / PRODUCT BLUEPRINT

DOCUMENT 06 OF 19

Audio Engine and DSP

A new touch-first performance instrument.
Professional audio, liquid glass and mechanical craft.



DESIGN HANDOFF BASELINE - PLANNING ONLY

29 October 2026 | Crystal Current selected; S26 Ultra test target

Independent of the previous Mixline preview. Targets are not measured results.

Version: 1.0 **Date:** 29 October 2026 **Status:** Design handoff baseline; implementation and validation pending **Product basis:** New product; no Mixline reuse. **Platform decision:** Android first is user-confirmed. **Accountable agents:** Audio Engine; Analysis/Stems; Android Platform. **Dependencies:** 05 Tech Stack and Architecture; 09 Quality and Device Validation; content-rights requirements.

Scope and audio authority

Implement a phone-native, sample-scheduled engine for two foundational music decks, expanding to the selected four music decks plus an independent sampler. The local version comes first and requires no AWS or home-lab server; future service work cannot become part of its render graph. Include manual mixing, cueing, scratching, tempo/pitch, key lock, sync, grid correction, loops, beat jump, slip, reverse, EQ/filter, FX, stems, monitoring and eligible recording. All numerical limits and budgets below are proposed targets; no engine or device has been tested in this planning session.

The audio engine is the authority for what is audible, when commands execute and what the interface displays. The owner selected ParDay armed assisted performance; preparation plus bounded transitions/automix are required capabilities, with explicit arming and immediate manual takeover. Galaxy S26 Ultra is the first test device, not a measured performance guarantee. ParDay never supplies the render clock or performs conversational inference in the callback. It produces proposals; only accepted, fresh, capability-checked actions become ordinary generation/epoch-bound engine commands. Preparation metadata may be undone, but audio already heard cannot be reversed by an Undo label. Optional assistant workloads yield before active audio and audible live-stem inference. Video follows the audio clock; broadcast encoders consume bounded off-thread taps. See Documents 13 and 15. Waveforms must represent real analyzed samples and actual playhead positions. A visual animation or timer must never stand in for working audio behavior. Each deck exposes an explicit state: empty, preparing, ready, playing, paused, seeking, ended or error, with separate flags for loop, slip, reverse and sync.

AE-01 - Decode, cache and supported media

Separate decoding from rendering. Background workers decode validated source media into bounded PCM pages with source-frame metadata. Oboe handles PCM I/O rather than compressed-file extraction/decoding. Android’s format table provides a platform baseline, but the product must publish a tested codec/container matrix, including actual bit depth, sample rate, channel handling, VBR and seeking behavior. [S1, S2]

Propose MP3, AAC/M4A, WAV and FLAC as initial required formats; evaluate AIFF, ALAC, Ogg Vorbis and Opus for the full product. Do not claim support solely from filename extensions. Unsupported DRM or providers without permitted PCM access remain ineligible for engine operations. A corrupted file must produce a clear error without replacing a working deck.

Cache policies must anticipate reverse and scratching around the playhead. Preload active cue/loop regions. Long seeks outside cache are asynchronous and visible; arbitrary compressed-file seeking cannot be promised instantaneous. Each load increments a generation identifier so a late decoder completion cannot insert an old track into a new deck. Acceptance: QA-07, QA-12 and QA-15.

AE-02 - Timing, tempo and musical operations

Use an output frame clock with monotonic host-time mapping and a stream epoch. Keep source position separate from output position because tempo, reverse, scratching and loops transform that relationship. The beatgrid maps musical beats to source frames and includes downbeat, tempo segments and manual corrections. Analysis confidence must be visible when unreliable; manual overrides take precedence over fresh automated analysis until intentionally reset.

Quantization schedules to engine frames using the selected subdivision. Sync uses the designated master deck/clock and its grid; loading a track must not unexpectedly change the master. Define separate

tempo-sync and phase-sync behaviors. Pitch bend is temporary; the tempo fader sets the persistent rate. Key lock changes time while preserving nominal pitch; key shift changes pitch independently within the validated range. Disable incompatible combinations explicitly instead of silently changing behavior.

Loop in/out points, half/double, roll, beat jump and saved loops require exact source positions. Loop jumps use controlled de-clicking. Slip maintains a shadow position so releasing scratch/reverse/roll returns to the expected musical point. Document whether a specific effect advances that shadow position; all UI labels must match. Acceptance: QA-05, QA-07 and QA-08.

AE-03 - DSP provider contract

The selected provider must support these operations or disclose the unsupported capability before selection:

Operation	Required contract
`prepare(config)`	Off-thread allocation; sample rate, maximum block, channels, quality and ranges
`latencyFrames()`	Exact current algorithmic latency and notification of permitted changes
`process(n, inputs, outputs)`	Bounded render into caller-owned buffers; no unexpected allocation/blocking
`applyParameters(frame, values)`	Defined sample/frame granularity, smoothing and safe range validation
`seek/reset(generation, sourceFrame)`	Defined flush, warm-up and timeline-reset behavior
`mapTimeline()`	Source-to-output mapping, including stretch/reverse and latency compensation
`capabilities()`	Supported combinations and restrictions, not merely individual features

Do not select a backend that cannot reconcile its internal time with the engine clock. Compensation is tested at varying tempo/key settings, not inferred from a fixed nominal delay. A provider that passes steady playback but drifts after parameter changes fails the relevant capability.

Evaluate integrated commercial DSP against modular libraries. Superpowered documents separate rate/pitch controls, quality options and mobile workload caveats at large rate changes. Rubber Band provides time/pitch processing under GPL/commercial routes. Their availability does not establish product quality; use a shared blind-listening and device benchmark. [S3, S4]

AE-04 - Mix graph and gain behavior

The proposed deck chain is source read/scratch → aligned prepared/live stem mix when active → time/pitch processing → trim → EQ/filter → assigned channel FX → channel fader → crossfader assignment → master sum. The sampler and eligible microphone paths enter documented buses. FX send/return placement and wet/dry behavior must be specified per effect so bypass does not change level unexpectedly.

Selected microphone input requires explicit permission and route selection, input gain/mute, a clear active indicator and a documented talkover/duck policy. Keep it muted until enabled. Verify input/output clock behavior, latency compensation and whether platform processing alters the signal. Do not promise feedback prevention or professional microphone quality from every built-in phone mic. A recording-only mode and live-monitor mode need distinct states; permission denial must leave music playback usable.

Define the headphone cue tap before channel fader/crossfader. Provide cue selection, cue level and cue/master blend. Preview/metronome routes must never leak into master. Master processing includes

headroom, optional master FX, limiter and metering. Meter pre/post points must be named. Record after the agreed master FX/limiter point and before physical headphone routing; hardware volume should not alter the recording.

Use floating-point internal processing and measured headroom. Apply parameter smoothing to gains, EQ/filter changes, cuts and routing transitions where musically appropriate. Specify linear/equal-power/cut crossfader curves with endpoint behavior and channel assignments. A limiter prevents output overflow within its validated operating conditions; it is not a substitute for useful trim and clipping indicators. Acceptance: QA-06, QA-08, QA-13 and QA-14.

AE-05 - Real-time execution and concurrency

No file/network I/O, database calls, memory allocation/free, mutex waits, sleeps, model inference or unbounded one-time calculations may occur in the device callback. Google specifically identifies blocking and bursty callback work as a dropout risk. [S5]

Preallocate buffers, pools and command capacity. Critical play/cue/load commands need sequence acknowledgements; continuous knob values may coalesce. Define queue saturation errors and stale-generation rejection. Control-side state never races directly with render-side mutable state. Route changes, provider initialization and recording finalization occur on control/worker threads.

Send bounded snapshots to the UI containing actual state, meters, position, stream epoch, underruns and capability status. Visual refresh cannot delay render. External library calls must be checked for real-time safety under the exact operation used. Acceptance: QA-04 and QA-07.

Scheduled automation uses an engine-owned automation ID/revision and engine instance guard, in addition to deck generation and stream epoch. The bounded event execution path checks these guards immediately before applying an event. Manual takeover invalidates affected pending automation before the manual command takes effect, so an already queued fader event cannot later reassert an old value. Stop/cancel, route/load invalidation and capability changes revoke the applicable revision; acknowledge cancellation and the last applied state honestly. Admit each native automation batch atomically or reject it entirely; never leave a partially scheduled transition hidden behind a success message. Trusted control-side code supplies this authority, never a model proposal. Cancellation has reserved bounded control capacity so a full normal queue cannot drop it; its acknowledgement names the effective frame after which affected events cannot execute. At a shared control boundary, invalidation precedes affected due events and manual values apply with ordinary smoothing. The plan's own accepted parameter writes do not invalidate its manual-control revision. Restart discards execution tokens and queued events; fresh arming is required.

AE-06 - Stems and sampler

Prepared four-part stems - vocals, drums, bass and other - are the baseline design target for eligible sources. Each stem set records source fingerprint, model and version, aligned sample count, channel format, preparation settings and integrity checks. Controls must distinguish unavailable, preparing, ready and failed. Full-mix fallback keeps the original source and user cue metadata intact.

Start with stem summing before one deck stretch stage unless individual stem DSP requires otherwise. If stems are processed independently, compensate all paths consistently and test reconstruction/phase alignment. Separation artifacts are a quality concern; do not describe outputs as exact original studio stems.

Live separation is conditional on a real mobile export and thermal/quality benchmark. Demucs is a research candidate whose upstream repository is archived; this is not a supported turnkey mobile product. Check exact code, weights and dependency licences separately. ExecuTorch and LiteRT are mobile-runtime candidates; model conversion and supported operations still need validation. [S6, S7, S8]

The sampler is separate from the four music decks. Specify one-shot/loop/gate modes, quantized trigger, velocity/level behavior, choke groups and voice-stealing order. Proposed certified profiles start with eight simultaneous voices and evaluate sixteen in the extended profile. Reserve voice capacity so a new pad does not interrupt a music deck. Rights and provenance travel with sample packs. Acceptance: QA-09, QA-13 and QA-18.

The selected sequencer adds editable step patterns, per-step velocity and mute, pattern banks, length/resolution, swing and quantized start/stop/pattern changes. Proposed initial grid: 16 steps per bar with selectable 1-4-bar patterns and explicit straight/triplet resolution where supported; final limits remain configurable acceptance decisions. Store event times in musical units, then schedule against the authoritative beat/engine clock. Edits during playback become a new pattern revision at the selected next bar boundary, with immediate mute/stop as distinct controls. A tempo or leader change remaps future events without double-triggering or altering already sounded events. Preserve sample and pattern identity through bank changes; voice limits and choke groups still apply. Validate repeated playback, swing bounds, tempo changes, stop/cancel and manual takeover under QA-05, QA-07 and QA-13.

AE-07 - Output, recording and interruptions

Support explicit master-only stereo, split mono master/cue, and verified four-channel USB profiles. A standard duplicated headphone splitter does not produce independent cue audio. AAudio streams attach to a device and require actual negotiated configuration checks; endpoint routing is not automatically supplied by the API. [S9]

Prefer a single multichannel device clock for stereo master plus stereo cue. Bluetooth remains a convenience/preparation route because Oboe documentation lists it among routes that may not provide low-latency streams. Do not market universal accurate scratching over wireless audio. [S1]

Recorder disk writes consume a bounded ring off-thread. Full disk, permission loss or encoder failure stops recording visibly without blocking live audio. Use recoverable chunks/journals where practical, and an explicit incomplete-recording status. Recording eligibility is checked per content source, including mixed-source sets.

Audio focus loss, calls, route removal and process death are distinct events. Obey system focus policy. On unsafe route loss, pause/mute rather than rerouting cue/master blindly. A process crash stops audio; restore session metadata at next launch without claiming uninterrupted playback or automatically blasting sound. Acceptance: QA-10, QA-11 and QA-18.

Default route-loss state transitions

These proposed defaults apply to the first implementation and must be tested under QA-10. Never equate a UI-selected device with a verified audio-channel map.

Event / current topology	Immediate state	Recovery condition
Master device disappears, or channel topology becomes unknown	Mute output, pause deck transports safely, invalidate the old route and stream epoch; preserve session state	Reopen off-thread, verify channels and show route; explicit user resume before audible output
Cue endpoint alone disappears while a separately verified master path remains unchanged	Disable and mute cue, keep master only if unchanged master mapping is positively verified; show alert	Verify cue mapping before enabling it; never send cue material to master as fallback

Event / current topology	Immediate state	Recovery condition
Whole USB multichannel interface disconnects	Treat as lost master; pause/mute and retain state	No automatic phone-speaker fallback; verify the entire topology and require resume
New device appears or an old device reconnects	Keep existing safe state; inspect capabilities off-thread	Accept only a validated mapping; a device name match alone is insufficient
Sample rate/channel count changes without user request	Mark route invalid, mute/pause, establish a new epoch	Rebuild format/latency mapping off-thread; resume deliberately
Focus is lost or a call interrupts	Follow the OS focus rule and transition to the documented interrupted state	Default to deliberate resume after disruptive focus loss; do not promise continued audio against OS policy

If platform behavior reroutes audio before the app observes the event, that is a concrete certification risk. Capture actual outputs and exclude unsupported route combinations rather than claiming software prevents every transient leak.

AE-08 - Thermal policy and completion gate

When resources tighten, suspend optional analysis and queued or precomputation stem jobs and reduce decorative rendering first. Inference supplying currently audible stems is protected active audio work; do not suspend it or silently replace its outputs with a full mix. P3 must be certified for that sustained workload. Never silently change active tempo, key, stem alignment, limiter latency, cue topology or audio route. Deny new expensive feature activation when the certified profile cannot support it. If the already-active graph cannot survive the defined maximum-load test after non-audio reductions, the profile fails certification.

Reduced visual effects must preserve control locations, hit areas and current gestures. Musical operation cannot change because the glass surface was simplified.

Engine completion requires QA-01 through QA-11 plus applicable QA-13/14/18 results, a pinned dependency/licence manifest, reproducible test corpus and documented unsupported combinations. Unit tests alone cannot certify touch response, thermal stability, output routing or subjective audio quality. Agents must return evidence and remaining limitations, not a visual demo as proof.

Sources

Checked 29 October 2026. Engineering targets and graph design are proposals.

- **S1:** Google, Oboe FAQ.
- **S2:** Android, Supported media formats.
- **S3:** Superpowered, TimeStretching API.
- **S4:** Breakfast Quay, Rubber Band Library.
- **S5:** Google, Low latency audio.
- **S6:** Meta Research, Demucs repository.
- **S7:** PyTorch, ExecuTorch on Android.
- **S8:** Google, LiteRT overview.
- **S9:** Android NDK, AAudio.